

gift yet: the ability to separate design from content. Browsers could support CSS, which could manipulate all the non-interactive properties of the HTML elements – colour, size, font, borders – all the stuff it would be pointless to have JavaScript do.

And so, the foundation for the browser engine is complete. Simple, really.

The muddy pool

In an ideal world, Netscape and Microsoft would have implemented the DOM and JavaScript in the same way in both their browsers. The browser wars would have then centred around only features, and we wouldn't suffer the agony of sites that don't used web standards. Ah, well.

It all probably started with JScript, IE's JavaScript me-too. While it could understand some of Netscape's JavaScript, no other browser could understand JScript. Both companies came up with their own custom additions to the DOM standard – Netscape came up with "layers", much like Photoshop's layers, which could be turned on and off using JavaScript; Microsoft, on the other hand, added the `document.all` object, which let JavaScript (or JScript) select any object in the document with a simple `document.all["id"]`, rather than navigating through the object tree, which was what the W3C would have wanted it to do. Other, more frivolous things came, too. Netscape, likely playing a nasty practical joke on the world, introduced the `<blink>` tag, which made text, well, blink annoyingly. Microsoft threw the `<marquee>` tag at us, resulting in the less annoying side-scrolling text.

Perhaps the only good thing to come out of this rush to implement new features was IE's `XMLHttpRequest` object, which lay

the foundation for the AJAX we all know and love. Besides that, though, the web essentially went to pot.

But you've lived through this first browser war, and you've seen browsers approach more standard DOM implementations.

The importance of objects

In Object-oriented Programming (OOP), objects have two key characteristics: properties, which describe the object, and behaviours, which describe what the object does. These objects are built on blueprints, called classes. An object from the "button" class, for example, has a property called "value", which tells the browser engine what text to use on the button; and the `click()` behaviour, which simulates the effect of being clicked.

Browser developers get to make their own blueprints for these objects (though it's a better idea to stick to the DOM standard specified by the W3C), giving them the ability to assign whatever properties and behaviours to HTML elements that they choose.

So why bother?

With all the recent emphasis on web standards, we've (somewhat) come beyond the days when different DOM implementations caused sites to run fine in one browser, and completely crash in another. As a user, there's little likelihood that you'll run into major problems.

Today's browser wars still centre around which features the engines can offer, but rather than coming up with their own non-standard features, the developers

now race to implement features from the latest W3C standards.

This, however, doesn't mean that browser engines are becoming homogenous.

Behind the best

Perhaps the most important rivalry between browser engines is that of WebKit – which powers Safari, Chrome, and others – and Gecko, which powers Firefox.

WebKit is a fork of KHTML, which was designed to be a simple, light HTML engine that was standards compliant, and easy to maintain. It's been praised for its clean code and small memory footprint, both of which played key roles in what WebKit has become today – the browsing engine that can hop platforms really, really easily. WebKit is now the engine behind Safari on the iPhone, the S60 web browser on Nokia mobile phones, Android's web browser, and Google's Chrome. Developers are also using WebKit with the GTK (GIMP Toolkit) to develop rich internet applications in Linux.

Gecko, on the other hand, is so much more than just a simple HTML engine. It gives us the XML User Interface Language (XUL), which gives developers a way to write full-featured applications using simple XML, CSS and JavaScript. It makes Firefox extensions possible, and we all know how appealing those extensions are to Firefox

users. XUL also became the base for applications such as Thunderbird, Songbird, and the Komodo IDE. To Gecko's disadvantage,

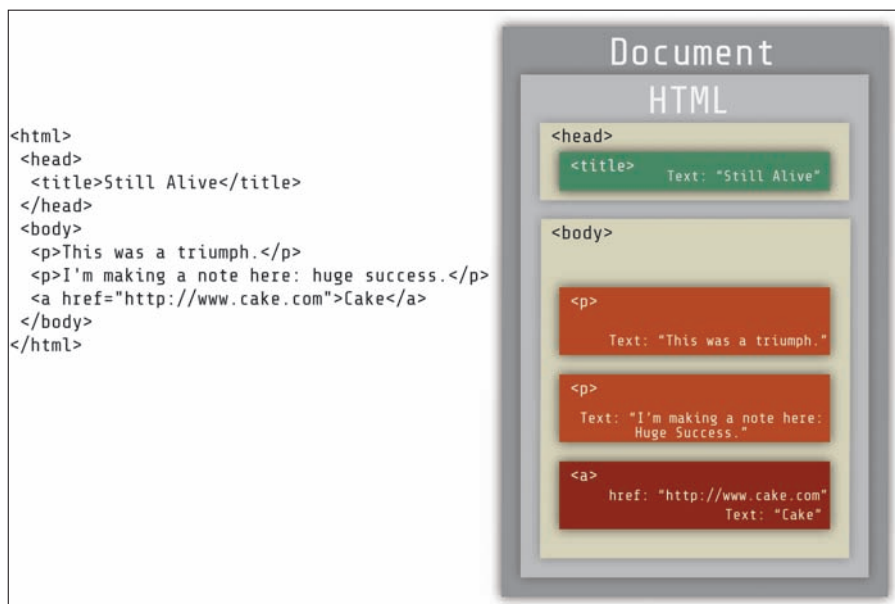
these features made the code bulky and complex, making it difficult for it to make the transition into the mobile world. After much toil, Mozilla's Gecko-based mobile browser – Fennec – is in beta (as of this writing), and has been well-received.

JScript gave Web developers many more features to play with, including the ability to store data

Keeping the engine running

It's simple economics, really: as customers, the ideal number of competitors we need is two, so that we can pit one against the other, and force them to offer us something better. WebKit may include better support for extensions, for example. We've already seen Gecko get speedier and lighter in response to competition from WebKit. If one browser engine were to dominate all our platforms, we'd just go back to the days of IE6 – innovation would stop, and every flaw in the engine would become painfully apparent on every platform, with no alternative in sight until the developers fixed it.

WebKit gives us rich internet applications as an added bonus, and Gecko gives us the ability to build desktop applications. Soon, we'll see more from both camps. This is good. This will do. **d**



Converting simple HTML code into a DOM